Lecture 01 – Optimization Algorithm

Stat 5376: Statistical Machine Learning

Ruiqi Liu

Department of Mathematics and Statistics
Texas Tech University

http://www.math.ttu.edu/~ruiqliu/

Disclaimer: These notes have not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.

Review of Linear Algebra

- Let $x = (x_1, \ldots, x_p)^\top \in \mathbb{R}^{p\times 1}$ and $A = (A_{ij}) \in \mathbb{R}^{p\times p}$.
- Linear Form:

$$Ax = \begin{pmatrix} \sum_{j=1}^p A_{1j}x_j \\ \vdots \\ \sum_{j=1}^p A_{pj}x_j \end{pmatrix}.$$

- Quadratic Form:

$$x^\top Ax = \sum_{i=1}^p \sum_{j=1}^p x_i x_j A_{ij}.$$

- Spectral Radius: For $A \in \mathbb{R}^{p\times p}$ and let $\lambda_1, \ldots, \lambda_p$ be the eigenvalues, then we define

$$\rho(A) = \max\{|\lambda_1|, \ldots, |\lambda_p|\}.$$

Derivative

- For a mapping $f : \mathbb{R}^k \to \mathbb{R}^p$, we denote

$$f(x) = \begin{pmatrix} f_1(x) \\ \vdots \\ f_p(x) \end{pmatrix} \text{ for } x \in \mathbb{R}^k.$$

- The derivative of f is

$$\dot{f}(x) = \frac{df(x)}{dx} = \begin{pmatrix} \frac{\partial f_1(x)}{\partial x_1} & \cdots & \frac{\partial f_1(x)}{\partial x_k} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_p(x)}{\partial x_1} & \cdots & \frac{\partial f_p(x)}{\partial x_k} \end{pmatrix} \in \mathbb{R}^{p \times k}.$$

- When $p = 1$, the derivative is a row:

$$\dot{f}(x) = \begin{pmatrix} \frac{\partial f(x)}{\partial x_1} & \cdots & \frac{\partial f(x)}{\partial x_k} \end{pmatrix} \in \mathbb{R}^{1 \times k}.$$

- Gradient: $\nabla f(x) = \dot{f}^\top(x) \in \mathbb{R}^{k \times 1}$ is a column.

Second Derivative

- When $p = 1$, the derivative is a row:

$$\dot{f}(x) = \begin{pmatrix} \frac{\partial f(x)}{\partial x_1} & \dots & \frac{\partial f(x)}{\partial x_k} \end{pmatrix} \in \mathbb{R}^{1 \times k}.$$

- When $p = 1$, we can take second derivative

$$\ddot{f}(x) = \frac{d\dot{f}^\top(x)}{dx} = \begin{pmatrix} \frac{\partial^2 f(x)}{\partial x_1^2} & \dots & \frac{\partial^2 f(x)}{\partial x_1 \partial x_k} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f(x)}{\partial x_k \partial x_1} & \dots & \frac{\partial^2 f(x)}{\partial x_k^2} \end{pmatrix} \in \mathbb{R}^{k \times k}.$$

- Hessian matrix: $\nabla^2 f(x) = \ddot{f}(x)$.

Taylor Expansion

- Recall: if $f : \mathbb{R} \to \mathbb{R}$, we have

$$f(x) = f(a) + f^{(1)}(a)(x-a) + \frac{1}{2!}f^{(2)}(a)(x-a)^2 + \frac{1}{3!}f^{(3)}(a)(x-a)^3 + \ldots.$$

- If $f : \mathbb{R}^d \to \mathbb{R}$, we have

$$f(x) = f(a) + (x-a)^\top \nabla f(a) + \frac{1}{2}(x-a)^\top \nabla^2 f(a)(x-a) + \ldots.$$

- For higher order term, we need tensor expression.

Chain Rule

- Why derivative is a row?
- $f : \mathbb{R}^k \to \mathbb{R}^p$ and $h : \mathbb{R}^p \to \mathbb{R}$.
- Composition: $g(x) = h(f(x)) = h(f_1(x), \ldots, f_p(x)) : \mathbb{R}^k \to \mathbb{R}$.
- Verify $\frac{\partial g}{\partial x_1} = \sum_{i=1}^p \frac{\partial h}{\partial f_i} \frac{\partial f_i}{\partial x_1}$
- Similarly $\frac{\partial g}{\partial x_j} = \sum_{i=1}^p \frac{\partial h}{\partial f_i} \frac{\partial f_i}{\partial x_j}$ for $j = 1, \ldots, k$.
- The derivative is $\dot{g} = \begin{pmatrix} \sum_{i=1}^p \frac{\partial h}{\partial f_i} \frac{\partial f_i}{\partial x_1} & \cdots & \sum_{i=1}^p \frac{\partial h}{\partial f_i} \frac{\partial f_i}{\partial x_k} \end{pmatrix} \in \mathbb{R}^{1 \times k}$.
- Notice that $\dot{h} = \begin{pmatrix} \frac{\partial h}{\partial f_1} & \cdots & \frac{\partial h}{\partial f_p} \end{pmatrix} \in \mathbb{R}^{1 \times p}$.
- And $\dot{f} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_k} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_p}{\partial x_1} & \cdots & \frac{\partial f_p}{\partial x_k} \end{pmatrix} \in \mathbb{R}^{p \times k}$.
- Chain rule: $\dot{g} = \dot{h}\dot{f} \in \mathbb{R}^{1 \times k}$.

Matrix Derivative

- Let $x = (x_1, \ldots, x_p)^\top \in \mathbb{R}^{p\times 1}$ and $A = (A_{ij}) \in \mathbb{R}^{q\times p}$.
- Consider $f(x) = Ax : \mathbb{R}^p \to \mathbb{R}^q$.
- By linear algebra, it follows that

$$f(x) = \begin{pmatrix} f_1(x) \\ \vdots \\ f_q(x) \end{pmatrix} = \begin{pmatrix} \sum_{j=1}^p A_{1j}x_j \\ \vdots \\ \sum_{j=1}^p A_{qj}x_j \end{pmatrix}.$$

- $\frac{\partial f_1(x)}{\partial x_1} = A_{11}$, $\frac{\partial f_1(x)}{\partial x_2} = A_{12}$, $\ldots$, $\frac{\partial f_1(x)}{\partial x_p} = A_{1p}$.
- Similarly, $\frac{\partial f_k(x)}{\partial x_1} = A_{k1}$, $\frac{\partial f_k(x)}{\partial x_2} = A_{k2}$, $\ldots$, $\frac{\partial f_k(x)}{\partial x_p} = A_{kp}$ for $k = 1, \ldots, p$.
- Derivative is

$$\dot{f}(x) = \begin{pmatrix} \frac{\partial f_1(x)}{\partial x_1} & \cdots & \frac{\partial f_1(x)}{\partial x_p} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_q(x)}{\partial x_1} & \cdots & \frac{\partial f_q(x)}{\partial x_p} \end{pmatrix} = \begin{pmatrix} A_{11} & \cdots & A_{1p} \\ \vdots & \ddots & \vdots \\ A_{q1} & \cdots & A_{qp} \end{pmatrix} = A \in \mathbb{R}^{q\times p}.$$

Matrix Derivative

- Let $x = (x_1, \ldots, x_p)^\top \in \mathbb{R}^{p\times 1}$ and $A = (A_{ij}) \in \mathbb{R}^{p\times p}$.
- Consider $f(x) = x^\top A x = \sum_{i=1}^p \sum_{j=1}^p A_{ij} x_i x_j : \mathbb{R}^p \to \mathbb{R}$.
- $f(x) = A_{11}x_1^2 + \sum_{i=2}^p A_{i1} x_i x_1 + \sum_{j=2}^p A_{1j} x_1 x_j + \text{ something}.$
- $\frac{\partial f(x)}{x_1} = 2A_{11}x_1 + \sum_{i=2}^p A_{i1}x_i + \sum_{j=2}^p A_{1j}x_j = \sum_{i=1}^p A_{i1}x_i + \sum_{j=1}^p A_{1j}x_j.$
- Similarly, for $k = 1, \ldots, p$, we have
$$\frac{\partial f(x)}{x_k} = \sum_{i=1}^p A_{ik}x_i + \sum_{j=1}^p A_{kj}x_j.$$
- Therefore, the derivative is

$$\begin{aligned}
\dot{f}(x) &= (\sum_{i=1}^p A_{i1}x_i + \sum_{j=1}^p A_{1j}x_j, \ldots, \sum_{i=1}^p A_{ip}x_i + \sum_{j=1}^p A_{pj}x_j) \\
&= (\sum_{i=1}^p A_{i1}x_i, \ldots, \sum_{i=1}^p A_{ip}x_i) + (\sum_{j=1}^p A_{1j}x_j, \ldots, \sum_{j=1}^p A_{pj}x_j) \\
&= (A^\top x)^\top + (Ax)^\top = x^\top (A + A^\top) \in \mathbb{R}^{1\times p}.
\end{aligned}$$

- If $A^\top = A$ is symmetric, then $\dot{f}(x) = 2x^\top A$.

Matrix Derivative

- Consider $f(x) = (c - Ax)^\top B(c - Ax)$.
- Let $h(x) = c - Ax$ and $g(h) = h^\top Bh$, so $f = g(h)$
- $\dot{h} = -A$ and $\dot{g} = h^\top(B^\top + B)$.
- So $\dot{f} = \dot{g}\dot{h} = -h^\top(B^\top + B)A = -(c - Ax)^\top(B^\top + B)A$.
- If $B = I$, $f(x) = (c - Ax)^\top(c - Ax)$, we can see
$$\dot{f}(x) = -2(c - Ax)^\top A.$$
- So the gradient is $\nabla f(x) = -2A^\top(c - Ax)$.
- Calculate $\nabla f(x) = 0$ leads to $A^\top Ax = A^\top c$.
- Solution is $x^* = (A^\top A)^{-1}A^\top c$.
- Linear regression: $\sum_{i=1}^n (y_i - x_i^\top \beta)^2 = (Y - X\beta)^\top(Y - X\beta)$, where

$$Y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^{n\times 1} \text{ and } X = \begin{pmatrix} x_1^\top \\ \vdots \\ x_n^\top \end{pmatrix} \in \mathbb{R}^{n\times p}.$$

Linear System

- In many applications, we need to solve the following linear system of equations

$$Ax^* = b, \text{ where } A \in \mathbb{R}^{p\times p} \text{ and } x^*, b \in \mathbb{R}^{p\times 1}.$$

- Here $x^* = A^{-1}b$ is the solution.
- Linear Regression: $f(\beta) = (Y - X\beta)^\top (Y - X\beta)$
- Gradient $\nabla f(\beta) = -2X^\top (Y - X\beta)$.
- The minimizer satisfies $X^\top X\beta^* = X^\top Y$.
- So the solution $\beta^* = (X^\top X)^{-1} X^\top Y$.

Iterative Algorithm

- A^{-1} is difficult to obtain.
- Consider decomposition $A = M - N$, with M^{-1} is easy to calculate.
- So $b = Ax = (M - N)x = Mx - Nx$ implies

$$x = M^{-1}Nx + M^{-1}b = M^{-1}(M - A)x + M^{-1}b = x + M^{-1}(b - Ax).$$

- Consider iteration: $x^{(k+1)} = x^{(k)} + M^{-1}(b - Ax^{(k)})$.
- Convergence if $\rho(M^{-1}N) < 1$.
- Intuition: M, N, A, b are all scalars, then

$$x^{(k+1)} = M^{-1}Nx^{(k)} + M^{-1}b \text{ is like geometric series.}$$

- Geometric series converge if $|r| = |M^{-1}N| < 1$.

Jacobi's Method

- $A = D - L - U$.
- D is a diagonal matrix.
- L is a lower triangular matrix.
- U is a upper triangular matrix.
- Example:

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}, \quad D = \begin{pmatrix} a_{11} & 0 & 0 \\ 0 & a_{22} & 0 \\ 0 & 0 & a_{33} \end{pmatrix}$$

- and

$$L = \begin{pmatrix} 0 & 0 & 0 \\ -a_{21} & 0 & 0 \\ -a_{31} & -a_{32} & 0 \end{pmatrix}, \quad U = \begin{pmatrix} 0 & -a_{12} & -a_{13} \\ 0 & 0 & -a_{23} \\ 0 & 0 & 0 \end{pmatrix}$$

Jacobi's Method

- $M = D$ and $N = L + U$.
- Advantage: M^{-1} is easy to calculate.
- Disadvantage: difficult to guarantee $\rho(M^{-1}N) < 1$.
- A Sufficient Condition for Convergence:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \text{ for all i .}$$

Relaxation Method

- For $w \in (0, 2)$, let $M = \frac{D}{w} - L$ and $N = \frac{1-w}{w} D + U$
- Notice M is a lower triangular matrix, its inverse can be easily calculated.
- Example: $\begin{pmatrix} a & 0 & 0 \\ b & c & 0 \\ d & e & f \end{pmatrix} \begin{pmatrix} \widetilde{a} & 0 & 0 \\ \widetilde{b} & \widetilde{c} & 0 \\ \widetilde{d} & \widetilde{e} & \widetilde{f} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- $a\widetilde{a} = 1 \Rightarrow \widetilde{a} = \frac{1}{a}$.
- $c\widetilde{c} = 1 \Rightarrow \widetilde{c} = \frac{1}{c}$.
- $b\widetilde{a} + c\widetilde{b} = 0 \Rightarrow \widetilde{b} = -\frac{b\widetilde{a}}{c} = -\frac{b}{ac}$.
- $f\widetilde{f} = 1 \Rightarrow \widetilde{f} = \frac{1}{f}$.
- $e\widetilde{c} + f\widetilde{e} = 0 \Rightarrow \widetilde{e} = -\frac{e\widetilde{c}}{f} = -\frac{e}{cf}$.
- $d\widetilde{a} + e\widetilde{b} + f\widetilde{d} = 0 \Rightarrow \widetilde{d} = -\frac{d\widetilde{a}+e\widetilde{b}}{f} = -\frac{cd-be}{acf}$.

Relaxation Method

- Advantage: Easy to check $\rho(M^{-1}N) < 1$.
- A Sufficient Condition for Convergence: If A is symmetric, positive definite, and $w \in (0, 2)$, then the relaxation method converges.
- In many applications, A satisfies the above condition.
- Regression Problem: $X^\top X\beta^* = X^\top Y$ or $\frac{1}{n}X^\top X\beta^* = \frac{1}{n}X^\top Y$
- In random design, $x_1, \ldots, x_n$ are i.i.d. sampled from x.
- Notice

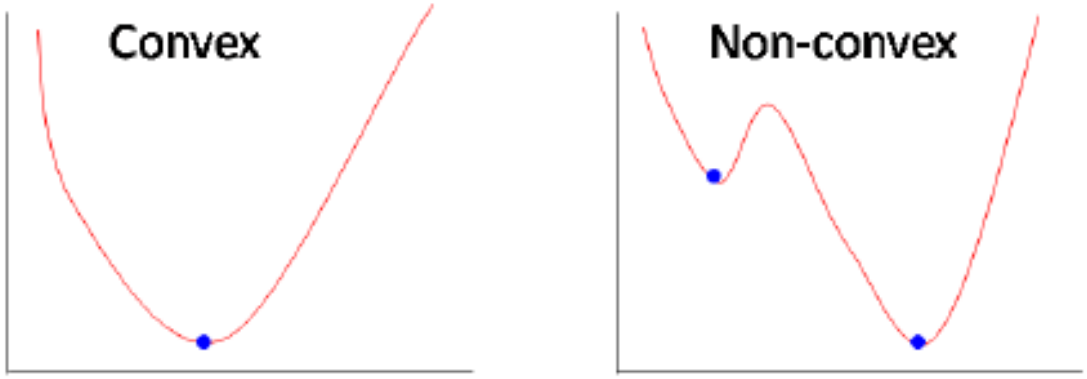
$$X = \begin{pmatrix} x_1^\top \\ \vdots \\ x_n^\top \end{pmatrix}, X^\top = \begin{pmatrix} x_1 & \ldots & x_n \end{pmatrix}, A = \frac{1}{n}X^\top X = \frac{1}{n}\sum_{i=1}^{n} x_i x_i^\top.$$

- By S.L.L.N, we can see $A \to E(xx^\top)$ almost surely as $n \to \infty$.
- If the eigenvalues of $E(xx^\top)$ are all greater than zero, then A is almost surely positive definite when sample size n is large.

Convexity

- A function $f(x)$ is called convex if

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2) \text{ for all } x_1, x_2 \text{ and } t \in [0,1].$$



- Convex function has a unique minimum.
- The local minimum is the global minimum.
- Convex function: $(x-1/2)^2$, e^x, $|x|$.
- Nonconvex function: $-(x-1/2)^2$, $\sin(x)$.

Subgradient

- u is said to be a subgradient of a function f at point x if

$$f(y) - f(x) \geq u_x^\top (y - x), \text{ for all } y.$$

- If f is differentiable, then $u_x = \nabla f(x)$ uniquely (It is a column.).
- If f is not differentiable, there could be many subgradients.
- $f(x) = |x|$, $u_x = \begin{cases} 1 & \text{if } x > 0; \\ -1 & \text{if } x < 0; \\ [-1, 1] & \text{if } x = 0. \end{cases}$
- **Proof:** Let Ω be the collection of subgradients of $f(x) = |x|$ at 0.
- If $u \in [-1, 1]$, we have $f(y) - f(0) = |y| \geq uy$ for all y.
- This shows $[-1, 1] \in \Omega$.
- If $u \in \Omega$, then by definition of subgradient and let $y = u$, we have

$$|u| = |y| = f(y) - f(0) \geq uy = u^2,$$

  which implies $|u| \leq 1$. This proves $\Omega \subset [-1, 1]$.

Subgradient

- $f(x) = \|x\|_2 = \sqrt{x^\top x}$, $u_x = \begin{cases} \frac{x}{\|x\|_2} & \text{if } x \neq 0; \\ \{z : \|z\|_2 \leq 1\} & \text{if } x = 0. \end{cases}$
- $f(x) = \|x\|_1 = \sum_{i=1}^p |x_i|$,

$$u_{x,i} = \begin{cases} 1 & \text{if } x_i > 0; \\ -1 & \text{if } x_i < 0; \\ [-1, 1] & \text{if } x_i = 0, \end{cases} \quad \text{and} \quad u_x = \begin{pmatrix} u_{x,1} \\ \vdots \\ u_{x,p} \end{pmatrix}.$$

- For simplicity, we still denote subgradient as $\nabla f(x)$ (It could be a set.).

Unconstrained Optimization

- Consider the optimization problem: $\min_{x \in \mathbb{R}^p} f(x)$.
- A saddle point $\widetilde{x}$ satisfy $0 \in \nabla f(\widetilde{x})$.
- $f(x) = \sin(x)$, $\nabla f(x) = \cos(x)$.
- Saddle points are $\{\ldots, -\frac{3\pi}{2}, -\frac{\pi}{2}, \frac{\pi}{2}, \frac{3\pi}{2}, \ldots\}$
- Global minimizers $x^* = \operatorname{argmin}_{x \in \mathbb{R}^p} f(x)$ are among the saddle points.
- $x^* = \{\ldots, -\frac{\pi}{2}, \frac{3\pi}{2}, \ldots\}$, multiple global minimizers.
- When f is convex, there is one saddle point, which is the global minimizer.
- $f(x) = \|x\|_1 = \sum_{i=1}^p |x_i|$,

$$u_{x,i} = \begin{cases} 1 & \text{if } x_i > 0; \\ -1 & \text{if } x_i < 0; \\ [-1, 1] & \text{if } x_i = 0, \end{cases} \quad \text{and} \quad \nabla f(x) = \begin{pmatrix} u_{x,1} \\ \vdots \\ u_{x,p} \end{pmatrix}.$$

- $0 \in \nabla f(x)$ implies $0 \in u_{x,i}$ for all $i = 1, \ldots, p$.
- Saddle points are $\{0\}$, which is also the global minimal.

Constrained Optimization

- Consider the optimization problem:

$$\begin{aligned} & \min_{x \in \mathbb{R}^p} f(x) \\ \text{subject to} \quad & h_i(x) \le 0, \text{ for } i = 1, \ldots, m, \\ & g_j(x) = 0, \text{ for } j = 1, \ldots, r. \end{aligned}$$

- Lagrange Multiplier: $\min_{x \in \mathbb{R}^p} f(x) + \sum_{i=1}^m u_i h_i(x) + \sum_{j=1}^r v_j g_j(x)$.
- Karush-Kuhn-Tucker (KKT) Conditions:
  Saddle points (or KKT points) $(x, u_i's, v_i's)$ satisfy
  - $0 \in \nabla f(x) + \sum_{i=1}^m u_i \nabla h_i(x) + \sum_{j=1}^r v_j \nabla g_j(x)$;
  - $u_i h_i(x) = 0$ for all i;
  - $h_i(x) \le 0, g_j(x) = 0$ for all i, j;
  - $u_i \ge 0$ for all i.
- If f, h_i's are convex, and $g_j(x) = a_j^\top x - b_j$ is linear, we say this is a convex problem.
  - Convex problem has one KKT point.
  - Nonconvex problem could have multiple KKT points.
  - Global minimizer is among the saddle points.

Exercise

- $\min_{x \in \mathbb{R}} -(x - 1/2)^2$ subject to $x \in [-1, 1]$.
- $f(x) = -(x - 1/2)^2$ and $h(x) = |x| - 1$ (no g).
- KKT Conditions:
  - $0 \in -2x + 1 + u\nabla h(x)$;
  - $u(|x| - 1) = 0$;
  - $|x| - 1 \leq 0$;
  - $u \geq 0$.
- Recall $\nabla h(x) = \begin{cases} 1 & \text{if } x > 0; \\ -1 & \text{if } x < 0; \\ [-1, 1] & \text{if } x = 0. \end{cases}$ We consider three cases.

Exercise

- Case 1: $x > 0$.
- KKT Conditions:
  - $0 \in -2x + 1 + u$;
  - $u(x-1) = 0$;
  - $x - 1 \leq 0$;
  - $u \geq 0$.
- $u = 0$ implies $x = 1/2$.
- $x = 1$ implies $u = 1$.
- Saddle Points: $\begin{cases} x = 1/2; \\ u = 0; \end{cases}$, $\begin{cases} x = 1; \\ u = 1. \end{cases}$

Exercise

- Case 2: $x < 0$.
- KKT Conditions:
  - $0 \in -2x + 1 - u$;
  - $u(-x - 1) = 0$;
  - $-x - 1 \leq 0$;
  - $u \geq 0$.
- $u = 0$ implies $x = 1/2$.
- $x = -1$ implies $u = 3$.
- Saddle Points: $\begin{cases} x = -1; \\ u = 3. \end{cases}$

Exercise

- Case 3: $x = 0$.
- KKT Conditions:
  - $0 \in -2x + 1 + u[-1, 1]$;
  - $u(-1) = 0$;
  - $-1 \leq 0$;
  - $u \geq 0$.
- $u = 0$ implies $x = 1/2$.
- No saddle points.

Exercise

- The saddle points (x, u) are $(1/2, 0), (1, 1), (-1, 3)$.
- Check $\min_{x \in \mathbb{R}} -(x - 1/2)^2$ at $x = 1/2, 1, -1$.
- Global minimizer $x^* = -1$.
- Multiple saddle points, because $f(x) = -(x - 1/2)^2$ is not convex.

Exercise

- Convex problem

$$\begin{aligned} &\min_{x \in \mathbb{R}^n} -\sum_{i=1}^{n} \log(\alpha_i + x_i) \\ \text{subject to} \quad & x_i \geq 0, \text{ for } i = 1, \ldots, n, \\ & \sum_{i=1}^{n} x_i = 1. \end{aligned}$$

- $f(x) = -\sum_{k=1}^{n} \log(\alpha_k + x_k)$, $h_i(x) = -x_i$, $g(x) = \sum_{i=1}^{n} x_i - 1$
- KKT Conditions:
  - $0 \in -\frac{1}{\alpha_i + x_i} - u_i + v$ for all i;
  - $u_i(-x_i) = 0$ for all i;
  - $-x_i \leq 0$ for all i and $\sum_{i=1}^{n} x_i - 1 = 0$;
  - $u_i \geq 0$ for all i.

Exercise

- KKT Conditions:
  - $0 \in -\frac{1}{\alpha_i + x_i} - u_i + v$ for all i;
  - $u_i x_i = 0$ for all i;
  - $x_i \geq 0$ for all i and $\sum_{i=1}^n x_i = 1$;
  - $u_i \geq 0$ for all i.
- Case 1: $u_i = 0$
- $0 = -\frac{1}{\alpha_i + x_i} - u_i + v$ implies $x_i = \frac{1}{v} - \alpha_i$;
- We need $x_i \geq 0$, so $v \leq \frac{1}{\alpha_i}$.
- Case 2: $u_i > 0$
- $x_i = 0$ and $0 \in -\frac{1}{\alpha_i + x_i} - u_i + v$ imply $u_i = v - \frac{1}{\alpha_i}$.
- We need $u_i > 0$, so $v > \frac{1}{\alpha_i}$.
- $x_i = \begin{cases} \frac{1}{v} - \alpha_i & \text{if } v \leq \frac{1}{\alpha_i}; \\ 0 & \text{if } v > \frac{1}{\alpha_i}; \end{cases} = \max(0, \frac{1}{v} - \alpha_i)$.

Exercise

- Using $\sum_{i=1}^n x_i = 1$, we need $\sum_{i=1}^n \max(0, \frac{1}{v} - \alpha_i) = 1$.
- Change of variable $y = \frac{1}{v}$
- $\sum_{i=1}^n \max(0, y - \alpha_i) = 1$ is a piecewise linear function of y.
- For example: assuming $\alpha_1 \leq \alpha_2 \leq \ldots \leq \alpha_n$.
- It is a piecewise linear function in $[0, \alpha_1], [\alpha_1, \alpha_2], \ldots, [\alpha_{n-1}, \alpha_n], [\alpha_n, \infty)$.
- You can find the solution y^* by checking each interval.
- Next calculate $v^* = \frac{1}{y^*}$.
- So $x_i^* = \max(0, \frac{1}{v^*} - \alpha_i) = \max(0, y^* - \alpha_i)$.
- Example: $n = 3, \alpha_1 = 1, \alpha_2 = 2, \alpha_3 = 3$
  - $\max(0, y - 1) + \max(0, y - 2) + \max(0, y - 3) = 1$ gives $y^* = 2$, $v^* = 0.5$
  - $x_1^* = 1$, $x_2^* = 0$, $x_3^* = 0$.
  - Optimal value is $-\log(1 + 1) - \log(2) - \log(3) = -\log(12) = -2.484907$

CVXR/CVXPY

- CVX is a package for solving convex optimization problem.

```
rm(list=ls())
library(CVXR)
n=3
x=Variable(n)
alpha=c(1,2,3)
obj=-1*sum(log(alpha+x))
constr=list(x>=0,sum(x)==1)
prob=Problem(Minimize(obj), constr)
result=solve(prob)
```

```
> result$value
[1] -2.484907
> result$getValue(x)
             [,1]
[1,] 9.998993e-01
[2,] 1.006791e-04
[3,] 3.483607e-08
> result$getDualValue(constr[[1]])
             [,1]
[1,] 6.771686e-09
[2,] 6.906044e-05
[3,] 1.667105e-01
> result$getDualValue(constr[[2]])
[1] 0.5000438
```

Primal and Dual

- Primal problem:

$$
\begin{aligned}
& p^* = \min_{x \in \mathbb{R}^p} f(x) \\
\text{subject to} \quad & h_i(x) \le 0, \text{ for } i = 1, \ldots, m, \\
& g_j(x) = 0, \text{ for } j = 1, \ldots, r.
\end{aligned}
$$

- Lagrange Multiplier:

$$
L(x, u, v) = f(x) + \sum_{i=1}^{m} u_i h_i(x) + \sum_{j=1}^{r} v_j g_j(x),
$$

where $u = (u_1, \ldots, u_m)^\top, v = (v_1, \ldots, v_r)^\top$.

- Fact: $p^* = \min_x \max_{u \ge 0, v} L(x, u, v)$

Primal and Dual

- Fact: $p^* = \min_x \max_{u \geq 0, v} L(x, u, v)$
- Proof: Case 1: If x is feasible, then $h_i(x) \leq 0$ and $g_j(x) = 0$ for all i, j. We have $\max_{u \geq 0, v} L(x, u, v) = f(x)$.
- Case 2: If x is not feasible, there is a i such that $h_i(x) > 0$ or a j such that $g_j(x) \neq 0$. For simplicity, we assume that $h_1(x) > 0$. We have

$$\max_{u \geq 0, v} L(x, u, v) \geq f(x) + \max_{u_1 \geq 0} u_1 h_1(x),$$

  which can be arbitrarily large. Hence, We have $\max_{u \geq 0, v} L(x, u, v) = \infty$.
- Combining above, we have

$$\max_{u \geq 0, v} L(x, u, v) = \begin{cases} f(x) & \text{if } h_i(x) \leq 0, g_j(x) = 0 \text{ for all } i, j, \\ \infty & \text{else.} \end{cases}$$

- Taking minimum over x, we complete the proof.

Primal and Dual

- Primal problem: $p^* = \min_x \max_{u \geq 0, v} L(x, u, v)$
- Define: $\phi(u, v) = \min_x L(x, u, v)$
- Dual problem: $d^* = \max_{u \geq 0, v} \phi(u, v) = \max_{u \geq 0, v} \min_x L(x, u, v)$
- Weak duality: $d^* \leq p^*$
- Proof: By definition, we have

$$\min_x L(x, u, v) \leq L(x, u, v)$$

$$\max_{u \geq 0, v} \min_x L(x, u, v) \leq \max_{u \geq 0, v} L(x, u, v)$$

$$\max_{u \geq 0, v} \min_x L(x, u, v) \leq \min_x \max_{u \geq 0, v} L(x, u, v)$$

- maxmin is less than minmax
- Weak duality always holds for convex or nonconvex problems.

Exercise

- Primal problem: $p^* = \min_x -(x-1/2)^2$ subject to $x \in [-1, 1]$.
- From previous slides, we have $p^* = -9/4$ when $x = -1$.
- Reformulation: $p^* = \min_x -(x-1/2)^2$ subject to $x - 1 \leq 0, -x - 1 \leq 0$.
- $L(x, u) = -(x-1/2)^2 + u_1(x-1) + u_2(-1-x)$ and

$$\phi(u) = \min_x \left\{-(x-1/2)^2 + u_1(x-1) + u_2(-1-x)\right\} = -\infty.$$

- Dual problem: $d^* = \max_{u \geq 0} \phi(u) = -\infty$.
- So, we have $d^* < p^*$ due to nonconvexity of the primal problem.

Exercise

- Primal problem: $p^* = \min_x (x - 1/2)^2$ subject to $x \in [-1, 1]$.
- It is easy to see that $p^* = 0$ when $x = 1/2$.
- Reformulation: $p^* = \min_x (x - 1/2)^2$ subject to $x - 1 \leq 0, -x - 1 \leq 0$.
- $L(x, u) = (x - 1/2)^2 + u_1(x - 1) + u_2(-1 - x)$ and $\phi(u) = \min_{x \in \mathbb{R}} L(x, u)$.
- Solving $0 = \frac{\partial}{\partial x} L(x, u) = 2(x - 1/2) + u_1 - u_2 = 2x - 1 + u_1 - u_2$ gives

$$x = x_u := \frac{u_2 + 1 - u_1}{2}.$$

- Hence, after some algebra, we have

$$\phi(u) = L(x_u, u) = -\frac{(u_2 - u_1)^2}{4} - \frac{u_1 + 3u_2}{2}$$

- Dual problem: $d^* = \max_u \phi(u)$ subject to $u_1, u_2 \geq 0$.
- So, we have $d^* = p^* = 0$ due to convexity of the primal problem.

Why Dual?

- Dual problem is always convex even if the primal problem is not.
- From dual to primal:
  - We wish to find the solution of the primal by solving the dual.
  - This requires the strong duality: $d^* = p^*$.
  - Strong duality holds for many problems: convex or nonconvex
  - Nonconvex: strong duality is verified case by case (hard!)
  - Convex: next slides
- If Strong duality holds, we have

$$\max_{u \geq 0, v} \underbrace{\min_x L(x, u, v)}_{\phi(u,v)} = \min_x \max_{u \geq 0, v} L(x, u, v).$$

  - Dual solution: $(u^*, v^*) = \operatorname{argmax}_{u \geq 0, v} \phi(u, v)$.
  - Primal solution: $x^* = \underbrace{\operatorname{argmin}_x L(x, u^*, v^*)}_{\text{unconstrained optimization}}$.

Exercise

- Primal problem: $p^* = \min_x (x - 1/2)^2$ subject to $x \in [-1, 1]$.
- Lagrange: $L(x, u) = (x - 1/2)^2 + u_1(x - 1) + u_2(-1 - x)$
- Dual problem: $d^* = \max_{u \geq 0} \phi(u)$ with

$$\phi(u) = -\frac{(u_2 - u_1)^2}{4} - \frac{u_1 + 3u_2}{2}$$

- Dual solution: $u^* = (0, 0)^\top$
- Primal solution:

$$x^* = \operatorname*{argmin}_x L(x, u^*) = \underbrace{\operatorname*{argmin}_x (x - 1/2)^2}_{\text{unconstrained optimization}} = 1/2.$$

Strong Duality of Convex Problem

- Primal problem:

$$\begin{aligned} & p^* = \min_{x \in \mathbb{R}^p} f(x) \\ \text{subject to} \quad & h_i(x) \le 0, \text{ for } i = 1, \ldots, m, \\ & a_j^\top x - b_j = 0, \text{ for } j = 1, \ldots, r. \end{aligned}$$

- Dual problem:

$$\begin{aligned} & d^* = \min_{u \in \mathbb{R}^m, v \in \mathbb{R}^r} \phi(u, v) \\ \text{subject to} \quad & u_i \ge 0, \text{ for } i = 1, \ldots, m. \end{aligned}$$

- Slater condition:
  - $h_i(x)$'s are convex.
  - There is a x such that $h_i(x) < 0$ for all i and $a_j^\top x = b_j$ for all j.
- Strong duality: Slater condition implies $p^* = d^*$ or equivalently

$$\max_{u \ge 0, v} \min_x L(x, u, v) = \min_x \max_{u \ge 0, v} L(x, u, v).$$

Quadratic Programming

- Equality-Constrained Quadratic Programming (EQP):

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \; & f(x) = \frac{1}{2} x^\top Q x + c^\top x \\ \text{subject to} \quad & Ax = b. \end{aligned}$$

- Here $Q \in \mathbb{R}^{n \times n}$ is symmetric, $c \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.
- Let $A = \begin{pmatrix} a_1^\top \\ \vdots \\ a_m^\top \end{pmatrix}$ and $b = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}$, so $Ax = \begin{pmatrix} a_1^\top x \\ \vdots \\ a_m^\top x \end{pmatrix} = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix} = b$

Quadratic Programming

- EQP can be written as

$$\begin{aligned} \min_{x \in \mathbb{R}^n} f(x) &= \frac{1}{2} x^\top Q x + c^\top x \\ \text{subject to} \quad & a_i^\top x = b_i \text{ for } i = 1, \ldots, m. \end{aligned}$$

- It is a convex problem.
- KKT Conditions:

$$\begin{aligned} 0 &= Qx + c + \sum_{i=1}^{m} \lambda_i a_i, \\ 0 &= a_i^\top x - b \text{ for } i = 1, \ldots, m. \end{aligned}$$

- Let $\lambda = (\lambda_1, \ldots, \lambda_m)^\top \in \mathbb{R}^m$, we can see

$$\begin{aligned} Qx + A^\top \lambda &= -c, \\ Ax - 0^\top \lambda &= b. \end{aligned}$$

Quadratic Programming

- The KKT point satisfies $\begin{pmatrix} Q & A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} x \\ \lambda \end{pmatrix} = \begin{pmatrix} -c \\ b \end{pmatrix}$.
- We can solve this problem using iterative algorithm.
- Example:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \; & f(x_1, x_2) = x_1^2 + 2x_2^2 \\ \text{subject to} \quad & x_1 + x_2 = 5. \end{aligned}$$

- $f(x_1, x_2) = \frac{1}{2} x^\top \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix} x$ and $(1, 1)x = 5$.
- So $Q = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}$, $c = 0$, $A = (1, 1)$ and $b = 5$.
- Solveing $\begin{pmatrix} 2 & 0 & 1 \\ 0 & 4 & 1 \\ 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \lambda \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 5 \end{pmatrix}$ leas to $x_1^* = \frac{10}{3}, x_2^* = \frac{5}{3}, \lambda^* = -\frac{20}{3}$.

Quadratic Programming

- General QP:

$$\begin{aligned} &\min_{x \in \mathbb{R}^n} f(x) = \frac{1}{2} x^\top Q x + c^\top x \\ \text{subject to} \quad &a_i^\top x = b_i \text{ for } i = 1, \ldots, m, \\ &a_j^\top x \leq b_j \text{ for } j = m+1, \ldots, m+p. \end{aligned}$$

- We say x is feasible if x satisfies the above constraints.
- Active set of x: $\mathcal{A}(x) = \{1, \ldots, m\} \cup \{j : a_j^\top x = b_j, j = m+1, \ldots, m+p\}$.

Active Set Algorithm (Optional)

- Idea: Choose an initial feasible $x_{(0)}$, let $W_0 = \mathcal{A}(x_{(0)})$ be its active set.
- We try to move $x_{(0)}$ to $x_{(0)} + d_{(0)}$ for some direction $d_{(0)}$.
- Finding $d_{(0)}$: Since $x_{(0)}$ is known, we can treat it as a constant, and consider

$$\begin{aligned} \min_{d \in \mathbb{R}^n} \; & f(d) = \frac{1}{2}(x_{(0)} + d)^\top Q (x_{(0)} + d) + c^\top (x_{(0)} + d) \\ \text{subject to} \quad & a_i^\top (x_{(0)} + d) = b_i \text{ for } i \in W_0. \end{aligned}$$

- Notice $f(d) = \frac{1}{2} d^\top Q d + (x_{(0)}^\top Q + c^\top) d + \text{ something free of } d.$
- Also $a_i^\top x_{(0)} = b_i$ as $i \in W_0$.
- The problem becomes

$$\begin{aligned} \min_{d \in \mathbb{R}^n} \; & f(d) = \frac{1}{2} d^\top Q d + (x_{(0)}^\top Q + c^\top) d \\ \text{subject to} \quad & a_i^\top d = 0 \text{ for } i \in W_0. \end{aligned}$$

- Let $d_{(0)}$ and λ^* be the solution of the above EQP.

Active Set Algorithm

- Moving: We consider several cases.
- Case 1: If $d_{(0)} \neq 0$, then
  - $a_i^\top (x_{(0)} + d_{(0)}) = b_i$ for all $i \in W_0$.
  - For some $i \notin W_0$, $x_{(0)} + d_{(0)}$ may violate the constraint $a_i^\top (x_{(0)} + d_{(0)}) \leq b_i$.
  - Try to move less: move $x_{(0)} + \gamma_{(0)} d_{(0)}$ for some $\gamma_{(0)} \in (0, 1]$.
  - How to choose $\gamma_{(0)}$?
  - For $i \notin W_0$, if $a_i^\top d_{(0)} \leq 0$, then you can choose any $\gamma_{(0)}$ because
  $$a_i^\top (x_{(0)} + \gamma_{(0)} d_{(0)}) = a_i^\top x_{(0)} + \gamma_{(0)} a_i^\top d_{(0)} \leq a_i^\top x_{(0)} \leq b_i.$$
  - For $i \notin W_0$, if $a_i^\top d_{(0)} > 0$, then we need find $\gamma_{(0)}$ satisfying
  $$a_i^\top (x_{(0)} + \gamma_{(0)} d_{(0)}) \leq b_i.$$
  - To do this, we need $\gamma_{(0)} \leq \frac{b_i - a_i^\top x_{(0)}}{a_i^\top d_{(0)}}$ for $i \notin W_0$ with $a_i^\top d_{(0)} > 0$.
  - Therefore, we choose
  $$\gamma_{(0)} = \min_{i \notin W_0, a_i^\top d_{(0)} > 0} \min(1, \frac{b_i - a_i^\top x_{(0)}}{a_i^\top d_{(0)}}).$$
  - Move $x_{(1)} = x_{(0)} + \gamma_{(0)} d_{(0)}$ and update $W_1 = \mathcal{A}(x_{(1)})$.

Active Set Algorithm

- Case 2: If $d_{(0)} = 0$, then we need to check the optimality.
  - Remember, the KKT conditions require the corresponding $\lambda_j^* \geq 0$ for inequality constraints.
  - If there are some $\lambda_j^* < 0$ for some inequality constraint, remove the index j with smallest λ_j^* from the Active set.
  - Move $x_{(1)} = x_{(0)}$ and update $W_1 = W_0 - \{j\}$.
- Repeat above procedures until $d_{(k)} = 0$ and the $\lambda_j^* \geq 0$ for all inequality constraints.

Active Set Algorithm

- Example:

$$\begin{aligned} &\min_{x\in\mathbb{R}^n} f(x_1,x_2) = x_1^2 + x_2^2 - 4x_1 - 4x_2. \\ \text{subject to} \quad & x_1 + x_2 \le 2, \\ & x_1 - 2x_2 \le 2, \\ & -x_1 - x_2 \le 1, \\ & -2x_1 + x_2 \le 2. \end{aligned}$$

- Matrix Form:

$$\begin{aligned} &\min_{x\in\mathbb{R}^n} f(x) = \frac{1}{2}x^\top \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} x + (-4,-4)x \\ \text{subject to} \quad & \begin{pmatrix} 1 & 1 \\ 1 & -2 \\ -1 & -1 \\ -2 & 1 \end{pmatrix} x \le \begin{pmatrix} 2 \\ 2 \\ 1 \\ 2 \end{pmatrix} \end{aligned}$$

- Choose $x_{(0)} = (0,-1)$, and $W_0 = \{2,3\}$.

Active Set Algorithm

- Original problem: $Q = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}, c = \begin{pmatrix} -4 \\ -4 \end{pmatrix}, A = \begin{pmatrix} 1 & 1 \\ 1 & -2 \\ -1 & -1 \\ -2 & 1 \end{pmatrix}, b = \begin{pmatrix} 2 \\ 2 \\ 1 \\ 2 \end{pmatrix}.$
- Choose $x_{(0)} = (0, -1)$, and $W_0 = \{2, 3\}$.
- Solve

$$\begin{aligned} \min_{d \in \mathbb{R}^n} f(d) &= \frac{1}{2} d^\top Q d + (x_{(0)}^\top Q + c^\top) d \\ \text{subject to} \quad & a_i^\top d = 0 \text{ for } i \in \{2, 3\}. \end{aligned}$$

- The problem becomes

$$\begin{aligned} \min_{d \in \mathbb{R}^n} f(d) &= \frac{1}{2} d^\top \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} d + \left[(0, -1) \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} + (-4, -4) \right] d \\ \text{subject to} \quad & \begin{pmatrix} 1 & -2 \\ -1 & -1 \end{pmatrix} d = 0. \end{aligned}$$

Active Set Algorithm

- Simplify to standard EQP:

$$\begin{aligned} \min_{d \in \mathbb{R}^n} f(d) &= \frac{1}{2} d^\top \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} d + (-4, -6) d \\ \text{subject to} & \quad \begin{pmatrix} 1 & -2 \\ -1 & -1 \end{pmatrix} d = 0. \end{aligned}$$

- The solution:

$$\begin{pmatrix} d_{(0)} \\ \lambda \end{pmatrix} = \begin{pmatrix} 2 & 0 & 1 & -1 \\ 0 & 2 & -2 & -1 \\ 1 & -2 & 0 & 0 \\ -1 & -1 & 0 & 0 \end{pmatrix}^{-1} \begin{pmatrix} 4 \\ 6 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ -0.667 \\ -4.667 \end{pmatrix}.$$

- Recall $W_0 = \{2, 3\}$, so $\lambda_2^* = -0.667$ and $\lambda_3^* = -4.667$.
- Remove 3 from the active set.
- $x_{(1)} = x_{(0)} = (0, -1)$ and update $W_1 = \{2\}$.

Active Set Algorithm

- Original problem: $Q = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}, c = \begin{pmatrix} -4 \\ -4 \end{pmatrix}, A = \begin{pmatrix} 1 & 1 \\ 1 & -2 \\ -1 & -1 \\ -2 & 1 \end{pmatrix}, b = \begin{pmatrix} 2 \\ 2 \\ 1 \\ 2 \end{pmatrix}.$
- Choose $x_{(1)} = (0, -1)$, and $W_1 = \{2\}$.
- Solve

$$\begin{aligned} \min_{d \in \mathbb{R}^n} f(d) &= \frac{1}{2} d^\top Q d + (x_{(1)}^\top Q + c^\top) d \\ \text{subject to} \quad & a_i^\top d = 0 \text{ for } i \in \{2\}. \end{aligned}$$

- The problem becomes

$$\begin{aligned} \min_{d \in \mathbb{R}^n} f(d) &= \frac{1}{2} d^\top \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} d + \left[(0, -1) \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} + (-4, -4) \right] d \\ \text{subject to} \quad & \begin{pmatrix} 1 & -2 \end{pmatrix} d = 0. \end{aligned}$$

Active Set Algorithm

- Simplify to standard EQP:

$$\begin{aligned} \min_{d \in \mathbb{R}^n} f(d) &= \frac{1}{2} d^\top \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} d + (-4, -6) d \\ \text{subject to} & \begin{pmatrix} 1 & -2 \end{pmatrix} d = 0. \end{aligned}$$

- The solution:

$$\begin{pmatrix} d_{(1)} \\ \lambda \end{pmatrix} = \begin{pmatrix} 2 & 0 & 1 \\ 0 & 2 & -2 \\ 1 & -2 & 0 \end{pmatrix}^{-1} \begin{pmatrix} 4 \\ 6 \\ 0 \end{pmatrix} = \begin{pmatrix} 2.8 \\ 1.4 \\ -1.6 \end{pmatrix}.$$

Active Set Algorithm

- $d_{(1)} = (2.8, 1.4)^\top$ and $\gamma_{(1)} = \min_{i \notin W_1, a_i^\top d_{(1)} > 0} \min(1, \frac{b_i - a_i^\top x_{(1)}}{a_i^\top d_{(1)}})$.
- $x_{(1)} = (0, -1)^\top$ and $W_1 = \{2\}$.
- Original problem: $Q = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}, c = \begin{pmatrix} -4 \\ -4 \end{pmatrix}, A = \begin{pmatrix} 1 & 1 \\ 1 & -2 \\ -1 & -1 \\ -2 & 1 \end{pmatrix}, b = \begin{pmatrix} 2 \\ 2 \\ 1 \\ 2 \end{pmatrix}$.
- $a_1^\top d_{(1)} = 4.2$, $a_3^\top d_{(1)} = -4.2$, $a_4^\top d_{(1)} = -4.2$
- $\gamma_{(1)} = \min(1, \frac{b_1 - a_1^\top x_{(1)}}{a_1^\top d_{(1)}}) = \frac{2+1}{4.2} = \min(1, \frac{5}{7}) = \frac{5}{7}$.
- Move $x_{(2)} = x_{(1)} + \gamma_{(1)} d_{(1)} = (2, 0)^\top$.
- We can verify $W_2 = \mathcal{A}(x_{(2)}) = \{1, 2\}$.
- Or a simpler method: $\gamma_{(1)}$ is blocked by $i = 1$, update $W_2 = \{1, 2\}$.

Active Set Algorithm

- $x_{(2)} = (2, 0)^\top, W_2 = \{1, 2\}$.
- Repeat to next step, we have $\begin{pmatrix} d_{(2)} \\ \lambda^* \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1.333 \\ -1.333 \end{pmatrix}$
- $\lambda_1^* = 1.333$ and $\lambda_2^* = -1.333$.
- Remove 2 from the active set.
- $x_{(3)} = x_{(2)} = (2, 0)^\top$ and update $W_3 = \{1\}$.

Active Set Algorithm

- $x_{(3)} = (2, 0)^\top, W_3 = \{1\}$.
- Repeat to next step, we have $\begin{pmatrix} d_{(3)} \\ \lambda^* \end{pmatrix} = \begin{pmatrix} -1 \\ 1 \\ 2 \end{pmatrix}$
- Original problem: $Q = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}, c = \begin{pmatrix} -4 \\ -4 \end{pmatrix}, A = \begin{pmatrix} 1 & 1 \\ 1 & -2 \\ -1 & -1 \\ -2 & 1 \end{pmatrix}, b = \begin{pmatrix} 2 \\ 2 \\ 1 \\ 2 \end{pmatrix}$.
- $\gamma_{(3)} = \min_{i \notin W_3, a_i^\top d_{(3)} > 0} \min(1, \frac{b_i - a_i^\top x_{(3)}}{a_i^\top d_{(3)}})$.
- $a_2^\top d_{(3)} = -3$, $a_3^\top d_{(3)} = 0$, $a_4^\top d_{(1)} = 3$
- $\gamma_{(3)} = \min(1, \frac{b_4 - a_4^\top x_{(3)}}{a_4^\top d_{(3)}}) = \min(1, \frac{2+4}{3}) = \min(1, 2) = 1$.
- Move $x_{(4)} = x_{(3)} + \gamma_{(3)} d_{(3)} = (1, 1)^\top$.
- It can be verify that $W_4 = \mathcal{A}(x_{(4)}) = \{1\}$.
- Or a simpler method: $\gamma_{(4)}$ is not blocked by any i, update $W_4 = W_3 = \{1\}$.

Active Set Algorithm

- $x_{(4)} = (1,1)^\top, W_3 = \{1\}$.
- Repeat to next step, we have $\begin{pmatrix} d_{(4)} \\ \lambda^* \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 2 \end{pmatrix}$
- $\lambda^* > 0$ and $d_{(4)} = 0$, the optimality conditions hold.
- So the solution is $x^* = (1,1)^\top$.

"quadprog" Package

```
library(quadprog)
Q=c(2,0,0,2)
Q=matrix(Q,ncol = 2, byrow = T)
c=c(-4,-4)
b=c(2,2,1,2)
A=c(1,1)
A=rbind(A, c(1,-2))
A=rbind(A, c(-1,-1))
A=rbind(A, c(-2,1))
```

```
> Q
     [,1] [,2]
[1,]    2    0
[2,]    0    2

> A
  [,1] [,2]
     1    1
     1   -2
    -1   -1
    -2    1

> c
[1] -4 -4
```

"quadprog" Package

```
solve.QP(Q,-c, -t(A), -b,meq=0)
```

```
$solution
[1] 1 1

$value
[1] -6

$unconstrained.solution
[1] 2 2

$iterations
[1] 2 0

$Lagrangian
[1] 2 0 0 0

$iact
[1] 1
```

Projection

- We say a set Ω is convex if

$$\lambda x + (1-\lambda)y \in \Omega \text{ for all } x, y \in \Omega \text{ and all } \lambda \in [0,1].$$

- Let Ω be a convex set and x be a point, which could be in or not in Ω.
- We say x^* is the projection of x onto Ω if

$$x^* \in \Omega \text{ and } \|x - x^*\| \leq \|x - y\| \text{ for all } y \in \Omega.$$

  Here $\|a\|^2 = a^\top a$ is the Euclidean norm.
- In other word, x^* is the closest point to x among all the points in Ω.
- The convexity of Ω guarantee the uniqueness of x^*.
- If $x \in \Omega$, then $x^* = x$.
- We write $x^* = Pr(x)$.

QP for Projection

- Consider convex set

$$\Omega = \{y : a_i^\top y = b_i \text{ for } i = 1, \ldots, m, a_j^\top y \leq b_j \text{ for } j = m+1, \ldots, m+p\}.$$

- The projection of x can be rewritten as

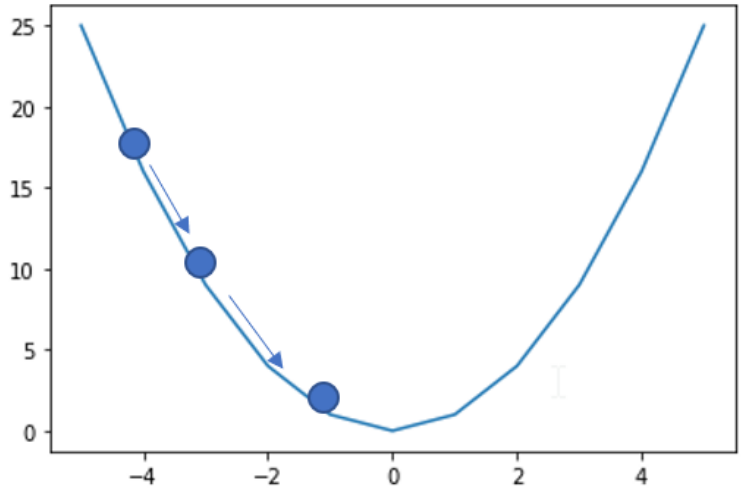
$$\begin{aligned} & \min_y f(y) = \|x - y\|^2 \\ \text{subject to} \quad & a_i^\top y = b_i \text{ for } i = 1, \ldots, m, \\ & a_j^\top y \leq b_j \text{ for } j = m+1, \ldots, m+p. \end{aligned}$$

- Notice $\|x - y\|^2 = y^\top y - 2x^\top y + x^\top x$.
- It is the equivalent to solve

$$\begin{aligned} & \min_y g(y) = \frac{1}{2} y^\top y - x^\top y \\ \text{subject to} \quad & a_i^\top y = b_i \text{ for } i = 1, \ldots, m, \\ & a_j^\top y \leq b_j \text{ for } j = m+1, \ldots, m+p. \end{aligned}$$

Gradient/Subgradient Descent

- Let f be a convex function and ∇f be one of its subgradient.
- Suppose x^* is the minimizer of f and $f^* = f(x^*)$ is the minimum.
- Algorithm: $x_{k+1} = x_k - \alpha_k \nabla f(x_k)$, here $\alpha_k > 0$ is the step size.
- Example:



- Choices of step size:
  - Constant step size: $\alpha_k = \alpha > 0$
  - Polyak's step size: $\sum_{k=0}^{\infty} \alpha_k = \infty$ and $\sum_{k=0}^{\infty} \alpha_k^2 < \infty$.
- Theoretically, x_0 can be chosen arbitrarily.
- The algorithm will stop when $|f(x_k) - f(x_{k+1})|$ or $\|\nabla f(x_k)\|$ is small.

Example

```
library(pracma)
f=function(x){
  s=1.2*(x-2)^2+3.2
  return(s)
}
gf=function(x){
  s=grad(f,x0=x)
  return(s)
}
gf=Vectorize(gf,"x")

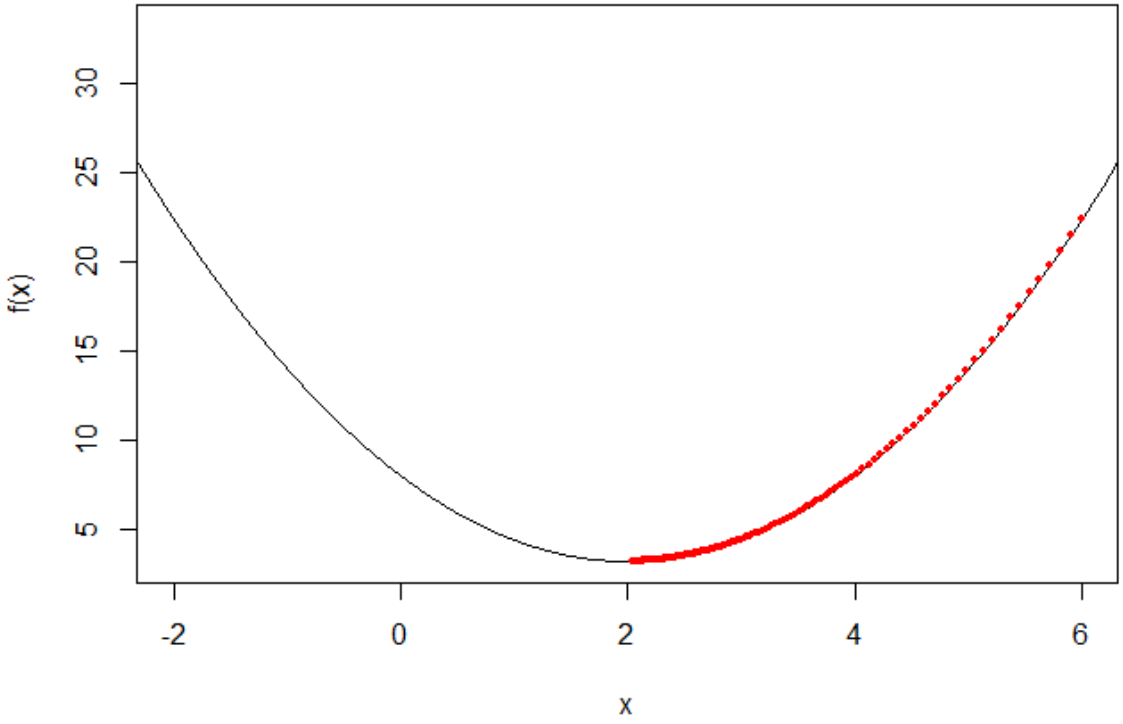
x=seq(-3,7,length.out=50)
plot(x,f(x),type="l",xlim=c(-2,6))

xk=6
alpha=0.01 #0.1, 0.6
error=1000
tol=10^(-4)
itmax=1000
it=1
while(it<=itmax & error>tol){
  points(xk, f(xk),pch=19,col="red",cex=0.5)
  xk_old=xk
  xk=xk-alpha*gf(xk)
  error=f(xk_old)-f(xk)
  error=abs(error)
  it=it+1
}
```

Example

- ```
  > xk
  [1] 2.040558
  ```

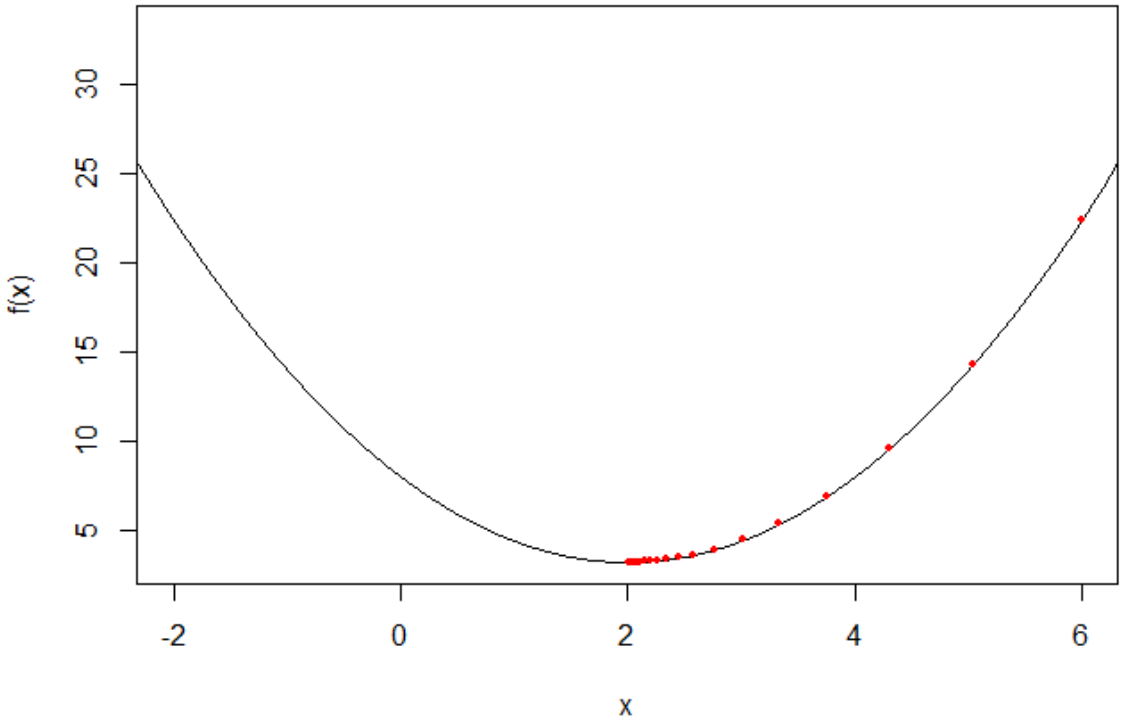
- Path with learning rate 0.01:


  ```

Example

- ```
  > xk
  [1] 2.009549
  ```

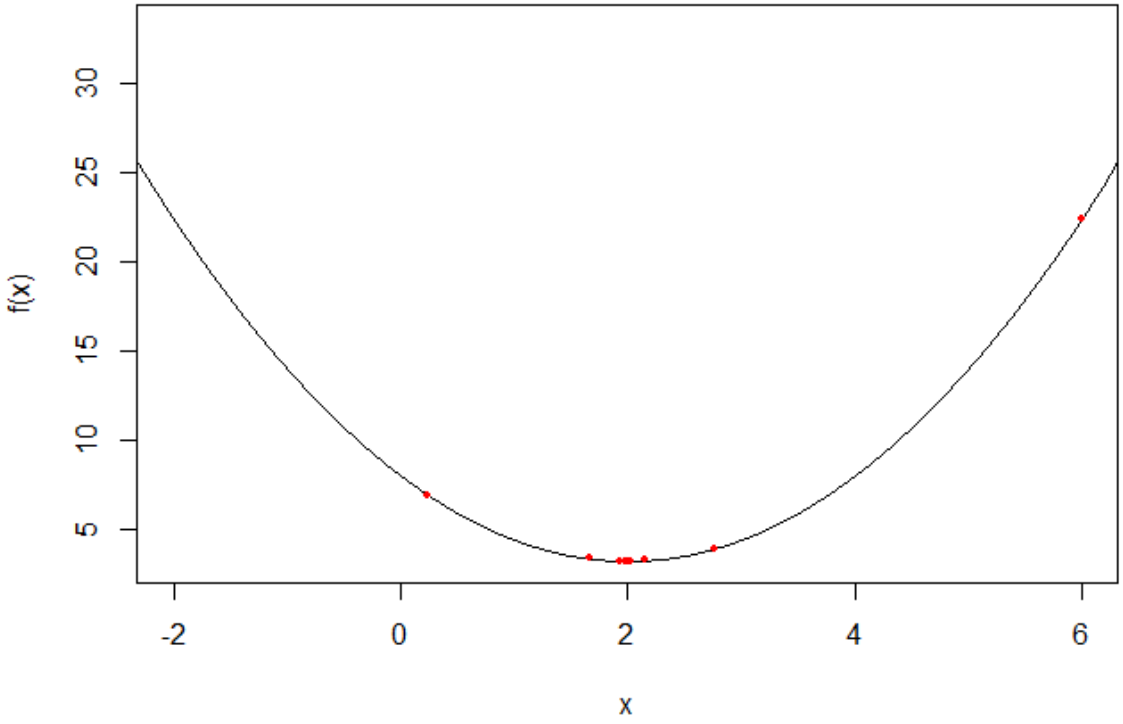
- Path with learning rate 0.1:


  ```

Example

- ```
  > xk
  [1] 1.997528
  ```

- Path with learning rate 0.6:


  ```

Projected Gradient/Subgradient Descent

- Let f be a convex function and ∇f be one of its subgradient.
- Let Ω be a convex set.
- Suppose x^* is the minimizer of f in Ω, that is

$$x^* = \operatorname*{argmin}_{x \in \Omega} f(x)$$

- Similarly, we define $f^* = f(x^*)$ to be the minimum.
- Algorithm: $x_{k+1} = Pr(x_k - \alpha_k \nabla f(x_k))$, here $\alpha_k > 0$ is the step size.
- The projection operator Pr forces x_k in Ω.

Optimization in statistical problems

- Statistical problem: $f(\beta) = \sum_{i=1}^{n} l(\beta, x_i, y_i)$, where (x_i, y_i) are observations.
- Linear regression: $f(\beta) = \sum_{i=1}^{n} (y_i - x_i^\top \beta)^2$.
- Logistic regression: $f(\beta) = \sum_{i=1}^{n} [\log(1 + e^{x_i^\top \beta}) - y_i x_i^\top \beta]$, with $y_i \in \{0, 1\}$.
- We can use gradient descent to solve the minimization problem.

Stochastic Gradient/Subgradient Descent

- If the objective function has the form $f(\beta) = \frac{1}{n}\sum_{i=1}^{n} l(\beta, x_i, y_i)$, we can use a stochastic algorithm.
- Stochastic Gradient Descent (SGD):
  - At the k-th iteration, randomly pick an index $i_k \in \{1, \ldots, n\}$
  - Updating $\beta_{k+1} = \beta_k - \alpha_k \nabla l(\beta_k, x_{i_k}, y_{i_k})$, here $\nabla l(\cdot, x_{i_k}, y_{i_k})$ is the subgradient of the function $\beta \to l(\beta, x_{i_k}, y_{i_k})$.
- Gradient Descent (GD): $\beta_{k+1} = \beta_k - \alpha_k \frac{1}{n}\sum_{i=1}^{n} \nabla l(\beta_k, x_i, y_i)$.
- GD uses all observations in each iteration, while SGD only uses one observation.
- In some literature, GD is called Batch Gradient Descent.
- Mini Batch Gradient Descent:
  - At the k-th iteration, randomly pick B indexes from $\{1, \ldots, n\}$ (with or without replacement).
  - Updating $\beta_{k+1} = \beta_k - \alpha_k \frac{1}{|B|}\sum_{s \in B} \nabla l(\beta_k, x_s, y_s)$,
- SGD is an example with $B = 1$, GD is the case with $B = n$.

Newton's method

- Goal: find $\operatorname{argmin} f(x)$ with f having second derivatives.
- Taylor expansion:

$$f(x) = f(a) + (x-a)^\top \nabla f(a) + \frac{1}{2}(x-a)^\top \nabla^2 f(a)(x-a) + \ldots$$

- Quadratic approximation:

$$f(x) \approx Q(x) = f(a) + (x-a)^\top \nabla f(a) + \frac{1}{2}(x-a)^\top \nabla^2 f(a)(x-a)$$

- Taking derivative

$$\nabla Q(x) = \nabla f(a) + \nabla^2 f(a)(x-a)$$

- $\nabla Q(x) = 0$ implies $x = a - (\nabla^2 f(a))^{-1} \nabla f(a)$
- Algorithm: $x_{k+1} = x_k - (\nabla^2 f(x_k))^{-1} \nabla f(x_k)$
- Gradient descent only uses first order condition of f, while Newton's method requires second differentiability.
- Unlike gradient descent, we do not need to rescale $f(x)$.

More Optimization Algorithms

- Adam (Adaptive Moment):
  - Most popular in pytorch
  - Use adaptively estimated first-order and second-order moments
- Proximal gradient method:
  - Achieve sparsity
  - Truth $(0, 1)$: sparse solution $(0, 0.9)$ V.S. nonsparse solution $(0.01, 1)$.
- DC (Difference of Convex) optimization:
  - $\min_x f(x) = f_1(x) - f_2(x)$, both $f_1(x), f_2(x)$ are convex.
  - $f(x)$ may not be convex.